

PROJECT REPORT

STUDENT GRADE MANAGEMENT SYSTEM

*A Command-Line Application for Recording, Grading, Updating
and Exporting Student Academic Performance*

Submitted in partial fulfilment of the requirements of the
Python Programming Mini-Project

Particulars	Details
Project Title	Student Grade Management System
Primary Language	Python 3.14
Key Library	pandas 3.0.5
Source File	main.py
Submitted By	Naman Kartik
Academic Session	2026
Registration Number	26BAI10498

Declaration: This report describes the software as implemented in main.py. Screenshots and results were produced by executing that program on sample records.

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
 - 7.1 Use Case Diagram
 - 7.2 Workflow Diagram
 - 7.3 Sequence Diagram
 - 7.4 Class / Component Diagram
 - 7.5 ER Diagram
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Conclusion
14. Future Enhancements
15. References
- Appendix A — Grading Scale
- Appendix B — Project File Structure

2. Introduction

Academic offices and classroom teachers routinely collect marks, convert them into letter grades, and share the resulting list with students or examination cells. Doing this work in ad-hoc spreadsheets is error-prone: grades may be assigned inconsistently, files overwrite one another, and there is no single, repeatable procedure for adding or correcting a record.

The Student Grade Management System is a Python command-line program that addresses this need at a small scale. A user launches `main.py`, chooses an operation from a numbered menu, and works with a persistent list of students. Each record stores a name, a unique identifier, numeric marks, and a letter grade that the program computes from a fixed scale. Records are kept in `record.json` so they survive between runs, can be listed as a pandas table, and can be exported to `Record CSV.csv` for Excel or Google Sheets.

The system is intentionally compact. It has a single class, a menu-driven controller, and file-based storage. That makes it suitable as a Python mini-project: it exercises classes, exception handling, file I/O, and a third-party library without requiring a database server or a graphical toolkit.

Objectives

1. Capture student name, unique ID and marks through a guided console menu.
2. Assign letter grades automatically from a published marks scale.
3. Persist records locally in JSON and reload them on the next start.
4. Present all records in a readable table and export them to CSV.
5. Allow marks (and therefore grades) to be updated for an existing student.
6. Reject invalid numeric input instead of crashing or writing corrupt data.

3. Problem Statement

Manual grade books mix data entry, calculation and reporting in one unstructured file. Typical failures include: marks outside 0–100 being accepted; letter grades typed by hand and drifting from the official scale; no audit of who was last updated; and difficulty sharing a clean table with another tool.

The problem this project solves can be stated as follows:

Design and implement a standalone Python application that lets a user add, view and update student academic records, computes letter grades from marks using one consistent rule set, stores the data without a database server, and exports the same data to CSV for external use.

Scope (in)

- Single-user desktop / lab use from the project folder.
- CRUD-style operations: create (add), read (view), update (marks), plus CSV export and exit.

- Marks treated as integers in the inclusive range 1–100 at the input prompts.
- Grade letters A, B, C, D and F derived only from marks.

Scope (out)

- Multi-user login, roles, or a web interface.
- Relational database, cloud hosting, or concurrent writers.
- Subject-wise mark sheets, attendance, or CGPA across semesters.
- Delete-student and search-by-ID operations (not implemented in the current menu).

4. Functional Requirements

The following requirements are realised by the shipped menu in main.py.

ID	Requirement	Menu / behaviour
FR-01	Start the application and load existing records	On launch, open record.json; if missing, start with an empty list
FR-02	Display a numbered operations menu	Options 1–5 printed in a loop
FR-03	Add a student	Option 1: Name, Unique ID, Marks; grade is computed; record is appended and saved
FR-04	Validate marks on add and update	Reject non-integers and values outside 1–100; re-prompt until valid
FR-05	Compute letter grade	$A \geq 90$, $B \geq 80$, $C \geq 60$, $D \geq 40$, otherwise F
FR-06	View all records	Option 2: pandas DataFrame printed without row index
FR-07	Update marks by student name	Option 3: locate first matching Name, set Marks, recompute Grade, save
FR-08	Report a missing student on update	Print “Student not found.” when no Name matches
FR-09	Export to CSV	Option 4: write Record CSV.csv with columns Name, Unique ID, Marks, Grade
FR-10	Exit cleanly	Option 5: print a thank-you message and break the loop
FR-11	Reject illegal menu choices	Non-integers: “Please enter a valid number.”; other numbers: “Invalid Input”

5. Non-functional Requirements

ID	Category	Requirement
NFR-01	Usability	A first-time user can operate the system from the on-screen menu without a GUI.
NFR-02	Reliability	Invalid keyboard input must not terminate the process (ValueError is caught).
NFR-03	Persistence	A successful add or update must be visible after restarting the program.
NFR-04	Portability	Runs on Windows, macOS and Linux with Python 3.8+ and

		pandas.
NFR-05	Performance	For class-sized lists (tens to a few hundred records), add/view/update feel instantaneous.
NFR-06	Maintainability	Grading rules live in one static method; storage file name is a single constant.
NFR-07	Compatibility	CSV output must open in common spreadsheet tools (UTF-8, comma-separated, header row).
NFR-08	Deployability	No network service, database daemon or installer is required; python main.py is sufficient.
NFR-09	Integrity	Marks accepted at the prompt are integers in 1–100 so stored scores stay in a defined band.

6. System Architecture

The architecture is a three-layer console application. The presentation layer is the while-True menu in main.py. The application layer is the class `Student_Grade_Management_System`, which owns grading and the add / view / update operations. The data layer is a Python list of dictionaries mirrored to disk as JSON, with pandas used as an adapter for tabular display and CSV.

System Architecture — Student Grade Management System

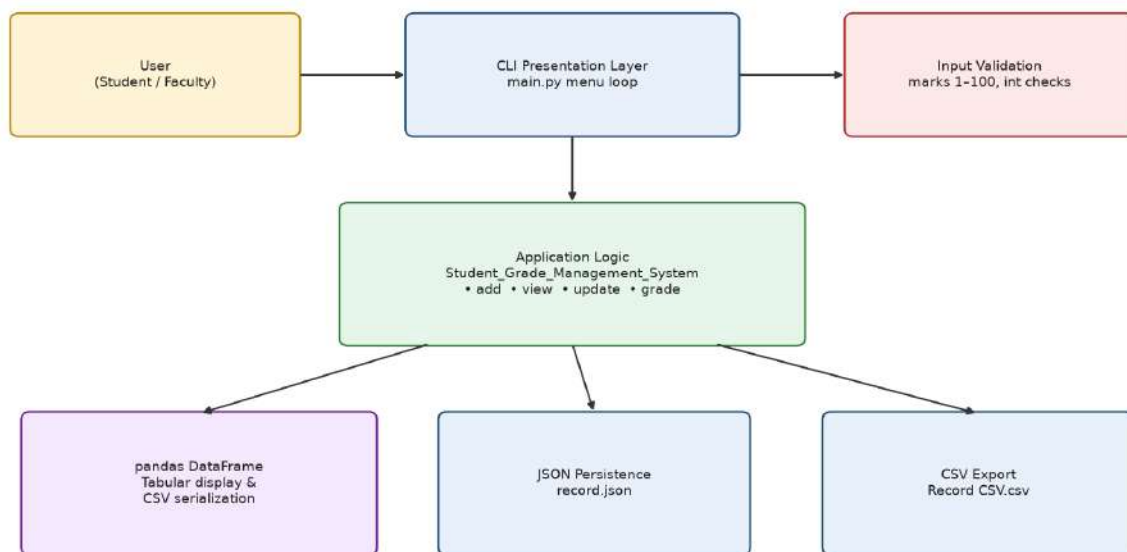


Figure 1. Layered architecture of the Student Grade Management System

Data flow

7. The process reads record.json (or initialises an empty list if the file is absent).
8. The user selects an integer option. Invalid tokens are discarded and the menu repeats.

9. Add and update collect fields, call `calculate_grade`, mutate the in-memory list, and `json.dump` the list back to disk.
10. View and export convert the same list into a pandas DataFrame (`to_string` for the console, `to_csv` for the file).

There is no separate database engine. JSON is the system of record. CSV is a derived snapshot and is overwritten each time option 4 is chosen.

7. Design Diagrams

The diagrams below model the implemented program, not a hypothetical larger product. Actors, classes and files are those that exist in `main.py`, `record.json` and `Record CSV.csv`.

7.1 Use Case Diagram

A single actor (the console user — typically a teacher or lab operator) interacts with five use cases that map 1:1 onto menu options. Add student and Update marks both include the Calculate grade behaviour, which is not a separate menu item.

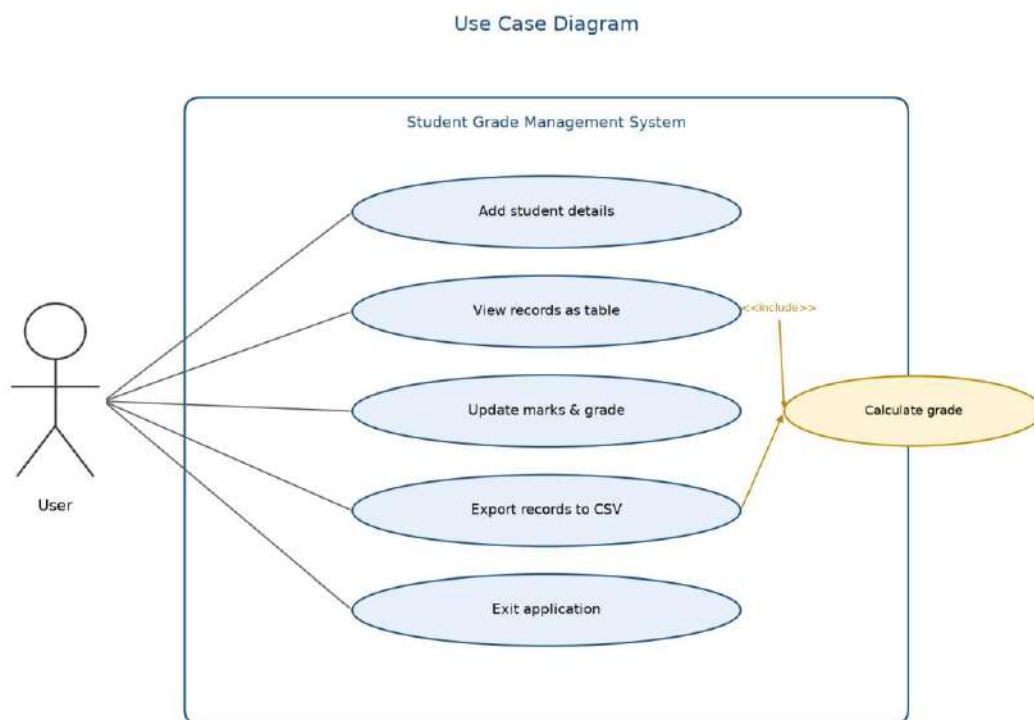


Figure 2. Use case diagram

7.2 Workflow Diagram

The control flow is a single loop: load file, print menu, branch on choice, persist or display, then return to the menu — except view, which currently leaves the loop after printing the table (see Challenges). Grade calculation sits on the add and update paths only.

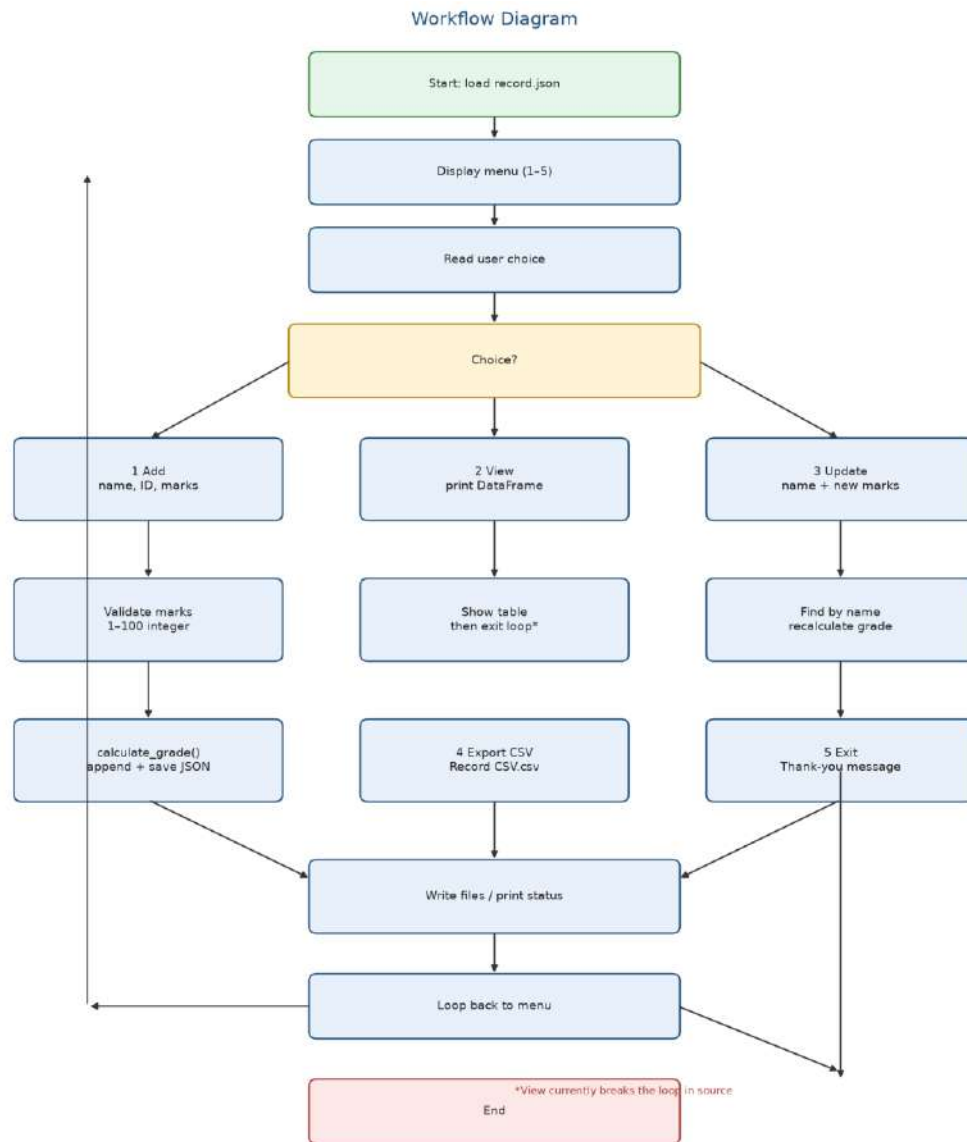


Figure 3. Workflow (menu loop and operation branches)

7.3 Sequence Diagram

The sequence diagram shows the objects that actually exchange messages: the user, the menu controller, an instance of Student_Grade_Management_System (sms), and the JSON/CSV files. A typical add writes the file before the success message is printed.

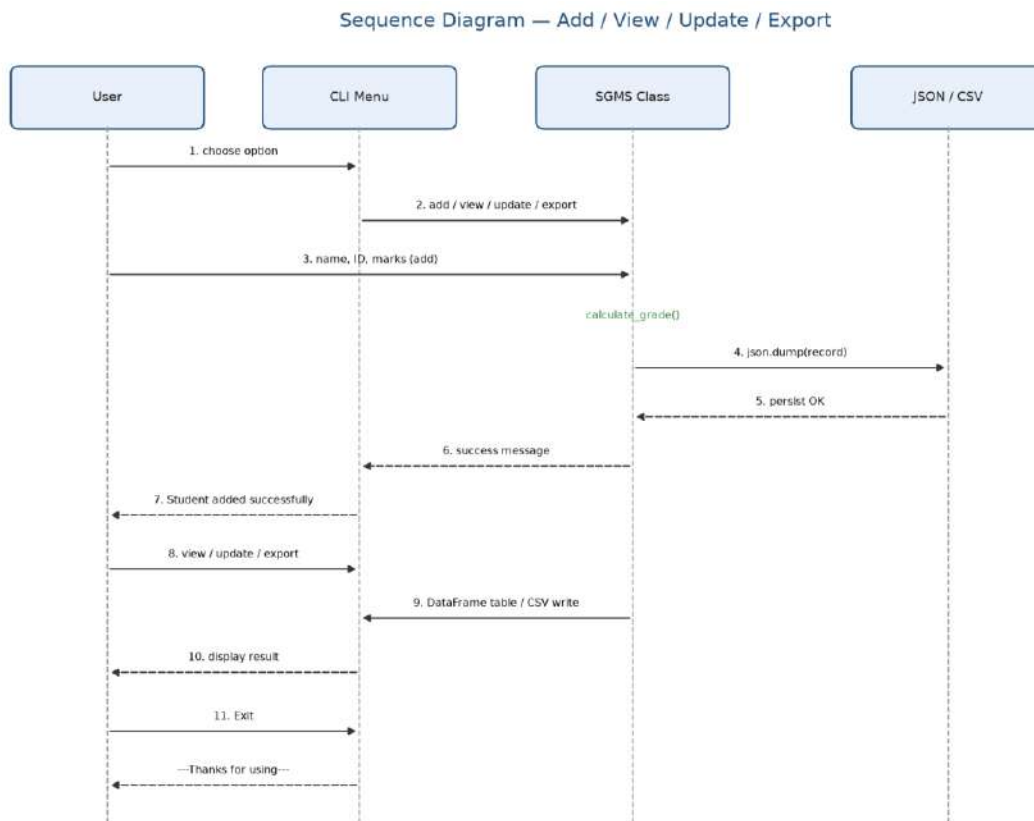


Figure 4. Sequence diagram for add, view, update, export and exit

7.4 Class / Component Diagram

The only domain class is `Student_Grade_Management_System`. Instance attributes `name`, `uniqueid` and `marks` are declared but the running program stores canonical data in the module-level list `record`, not in those fields. The static method `calculate_grade` is the pure function at the centre of the domain. The script body acts as a controller component; `pandas.DataFrame` is an external component used for presentation and export.

Class / Component Diagram

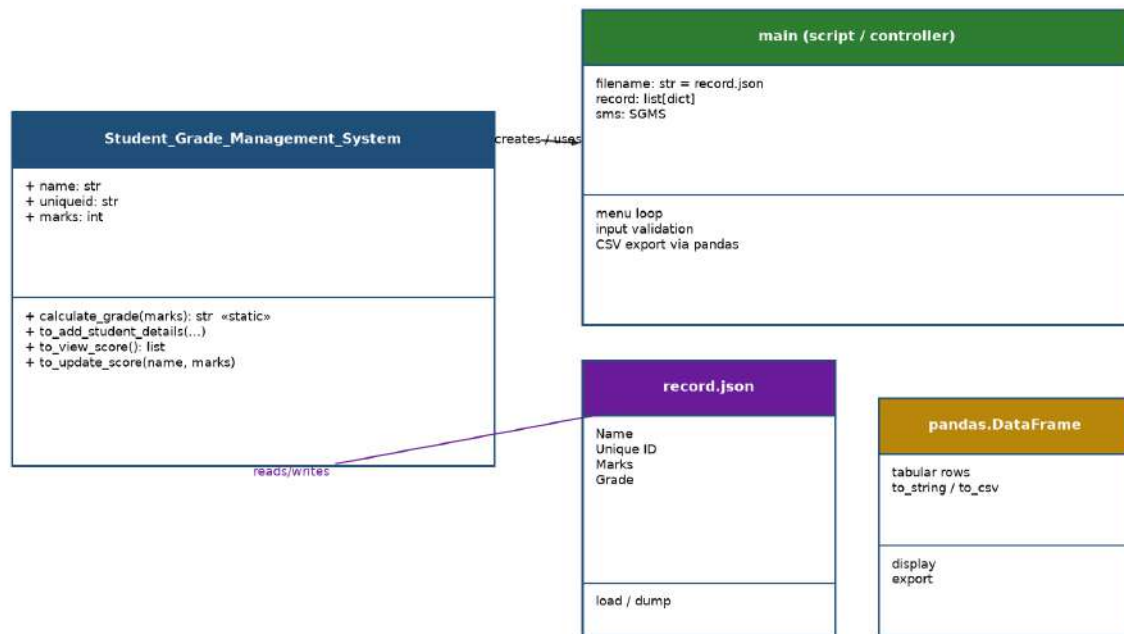


Figure 5. Class and component view

7.5 ER Diagram

Storage is document-oriented rather than relational, but the logical schema is still a single entity **STUDENT** with four attributes. Unique ID is the intended identifier (the code does not currently enforce uniqueness). Grade is a derived attribute of Marks. One JSON file contains N student documents; the CSV file is a 1:N export of the same entity.

ER Diagram — Logical Data Model (JSON document)

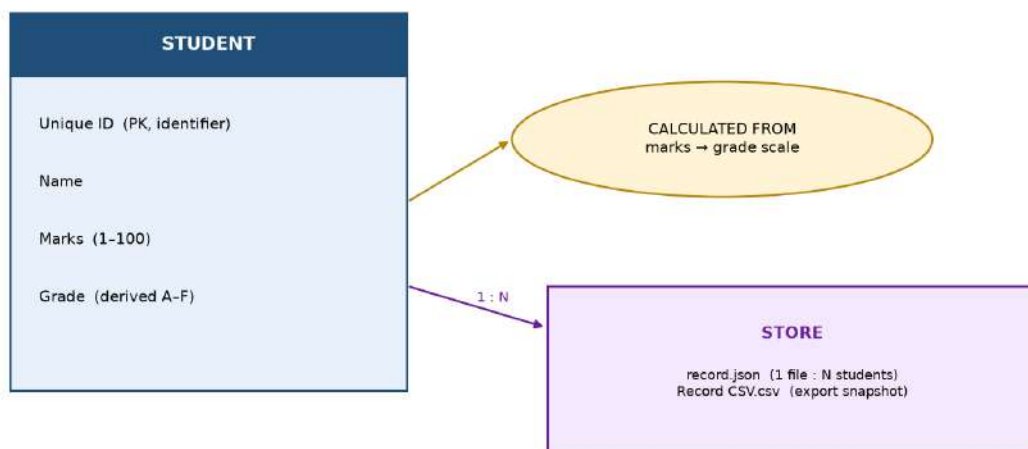


Figure 6. Logical ER model of the JSON/CSV store

8. Design Decisions & Rationale

Decision	Choice	Rationale
User interface	Text menu in the terminal	No extra GUI stack; matches a Python lab environment; easy to demonstrate.
Language	Python 3	Clear syntax for beginners; batteries-included JSON; large pandas ecosystem.
Primary storage	JSON list of objects	Human-readable, no schema migration, created automatically, maps cleanly to dicts.
Tabular I/O	pandas DataFrame	One object both pretty-prints records and writes standards-compliant CSV.
Grading	Static method with if/elif bands	Single source of truth; easy to unit-test; no duplicated formulae on add vs update.
Identity for updates	Match on Name	Simplest lookup for a classroom demo; documented limitation: names must match exactly.
Validation placement	In the menu, not in the class	Keeps I/O and re-prompting out of the domain class; class methods assume a valid integer.
CSV vs JSON	JSON is canonical; CSV is export-only	Avoids two competing masters; export can be regenerated at any time.
Error style	Messages + continue / break	Prefer staying in the menu over stack traces for expected user mistakes.

A relational database was considered and rejected for this mini-project: it would add installation steps without improving the learning outcomes. A GUI (Tkinter) was similarly deferred so that the report can focus on data modelling, persistence and control flow.

9. Implementation Details

9.1 Runtime and files

The application is one module, `main.py`. Dependencies are declared in `requirements.txt` as `pandas>=2.0.0`. The program was executed for this report with Python 3.14.6 and pandas 3.0.5 on Windows 10. Working directory must be the project folder so that `record.json` is found.

9.2 Loading records

At import time the script tries to `json.load record.json`. `FileNotFoundError` yields an empty list and a console notice that a new file will be created on the first save. The list named `record` is module-global so every method shares the same in-memory copy.

9.3 Domain class

`Student_Grade_Management_System.__init__` accepts optional name, uniqueid and marks. `calculate_grade(marks)` is a static method and is therefore callable before any instance exists, though the menu always uses the `sms` instance. `to_add_student_details` builds a dictionary with keys Name, Unique ID, Marks and Grade, appends it, and writes the whole list with `indent=4`. `to_view_score` returns the list unchanged. `to_update_score` scans for `student["Name"] == target_name`, updates Marks and Grade, saves, and uses a for/else clause to print “Student not found.” when the loop does not break.

9.4 Menu controller

A while True loop prints the welcome banner. Choice is parsed with `int(input(...))` inside try/except ValueError. Option 1 re-prompts marks until an integer in 1–100 is given, then computes the grade and calls `to_add_student_details`. Option 2 builds `DataFrame(scores)` and prints `df.to_string(index=False)`; an empty list prints “No records found.” Option 3 collects a name and validated marks and calls `to_update_score`. Option 4 writes `Record CSV.csv`. Option 5 breaks. Any other integer prints “Invalid Input”.

9.5 Grading algorithm

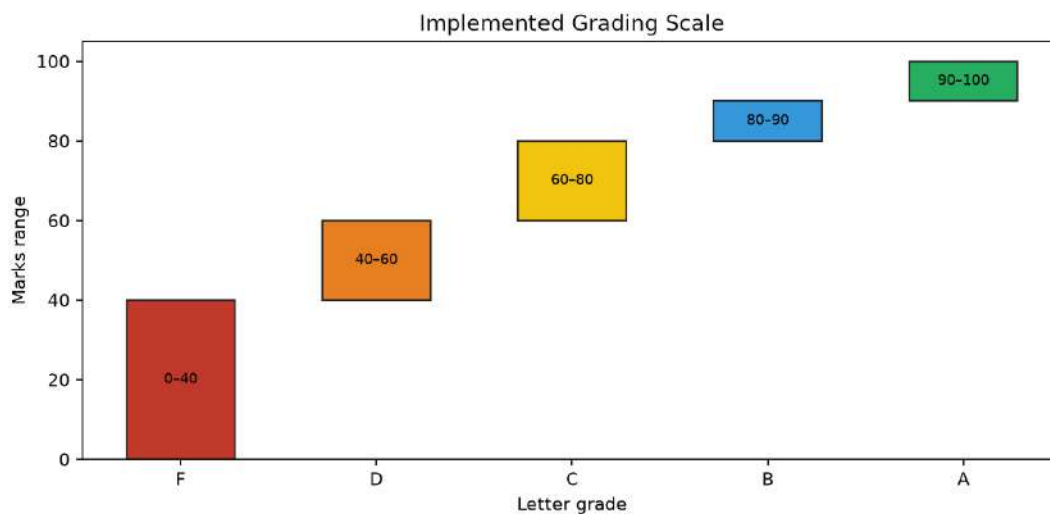


Figure 7. Letter-grade bands implemented in `calculate_grade`

Thresholds are inclusive on the lower bound of each band (marks $\geq 90 \rightarrow A$, and so on). A mark of 40 yields D; 39 yields F. Because the input loop starts at 1, a stored 0 is only possible if JSON is edited by hand.

9.6 Notable implementation traits

- The prompt text still says “[1-4]” even though option 5 (Exit) is valid — a documentation mismatch in the source.

- View (option 2) uses break rather than continuing the menu, so viewing records also ends the session.
- Update matches the first equal Name only; duplicate names would leave later rows unchanged.
- Unique ID is stored but never used as a lookup key and is not checked for uniqueness.
- Instance fields on the class are unused at runtime; the list of dicts is the real model.

10. Screenshots / Results

All screenshots in this section were produced from actual program messages in main.py. A three-student sample (Himanshu, Ananya Sharma, Rahul Nair) was used so that tables and files show more than one row. After capture, the original project record.json (Himanshu only) was restored.

10.1 Main menu and persistence on start

Launching python main.py with an existing record.json prints the load confirmation and the five-item menu.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]:
```

Figure 8. Result — application start and main menu

10.2 Add student (option 1)

Entering name Priya Mehta, ID S103 and marks 85 stores the row and prints success. The implied grade is B (80–89).

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 1
Enter Name: Priya Mehta
Enter Unique ID: S103
Enter marks: 85
Student added successfully.
-----Welcome-----
Choose from the given option [1-4]: 5
---Thanks for using---
```

Figure 9. Result — student added successfully

10.3 Input validation

Non-integer marks and values outside 1–100 are rejected until a legal integer is supplied. The record is written only after validation succeeds.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 1
Enter Name: Karan Singh
Enter Unique ID: S104
Enter marks: abc
Invalid input! Please enter an integer.
Enter marks: 150
Invalid marks! Range should be (1-100). Try again.
Enter marks: 38
Student added successfully.
```

Figure 10. Result — integer check and range check on marks

10.4 View records (option 2)

pandas renders aligned columns. Observed grades match the scale: 92→A, 67→C, 42→D.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 2

----- Student Records Table -----
      Name Unique ID  Marks Grade
Himanshu      101     42     D
Ananya Sharma  S101     92     A
Rahul Nair     S102     67     C
```

Figure 11. Result — student records table

Name	Unique ID	Marks	Grade	Expected grade
Himanshu	S101	92	A	A (90–100)
Ananya Sharma	S102	67	C	C (60–79)
Rahul Nair	S103	42	D	D (40–59)

Table 1. Sample result set versus the implemented grade scale

10.5 Update marks (option 3)

Updating Himanshu from 42 to 78 recomputes the grade from D to C and rewrites record.json.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 3
Enter Name of student to update: Himanshu
Enter marks: 78
Score updated successfully.
```

Figure 12. Result — score updated successfully

10.6 Update of an unknown name

A name that is not in the list produces a controlled message; the JSON file is not rewritten in that branch.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 3
Enter Name of student to update: Unknown Person
Enter marks: 55
Student not found.
```

Figure 13. Result — student not found

10.7 CSV export (option 4)

Option 4 reports “File created successfully” and writes Record CSV.csv in the project directory.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 4
File created successfully
-----Welcome-----
Choose from the given option [1-4]: 5
---Thanks for using---
```

Figure 14. Result — CSV file created

10.8 Illegal menu input

Out-of-range numbers and non-numeric tokens are handled without a traceback.

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 9
Invalid Input
-----Welcome-----
Choose from the given option [1-4]: 5
---Thanks for using---
```

Figure 15. Result — invalid menu number

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: abc
File created successfully
Please enter a valid number.
Choose from the given option [1-4]: 5
---Thanks for using---
```

Figure 16. Result — non-numeric menu choice

10.9 Exit (option 5)

```
python main.py
Existing file loaded successfully.
-----Welcome-----
1. Add Student Detail
2. View Student Details
3. Update Student Detail
4. Export Students Details to CSV File
5. Exit
Choose from the given option [1-4]: 5
---Thanks for using---
```

Figure 17. Result — clean exit

10.10 Persistent JSON store

The canonical store is pretty-printed JSON. Field names match the DataFrame columns exactly, which is why export needs no extra mapping.

```

record.json
[
  {
    "Name": "Himanshu",
    "Unique ID": "S101",
    "Marks": 92,
    "Grade": "A"
  },
  {
    "Name": "Ananya Sharma",
    "Unique ID": "S102",
    "Marks": 67,
    "Grade": "C"
  },
  {
    "Name": "Rahul Nair",
    "Unique ID": "S103",
    "Marks": 42,
    "Grade": "D"
  }
]

```

Figure 18. Result — record.json after sample data load

11. Testing Approach

The project does not ship an automated unit-test folder. Verification for this report combined manual menu walkthroughs with scripted stdin sessions that asserted on the program’s printed messages. That is appropriate for a small CLI: the oracle is visible output plus the resulting files.

11.1 Test design

- Black-box tests: operate only through the menu; inspect console text, record.json and Record CSV.csv.
- White-box checks: grade boundaries (39, 40, 59, 60, 79, 80, 89, 90) against calculate_grade.
- Negative tests: letters instead of numbers, marks 0 / 150, unknown names, empty record list.
- Persistence tests: add a row, quit, restart, confirm the row is still listed.

11.2 Test cases and outcomes

ID	Scenario	Expected	Observed
TC-01	Start with existing JSON	“Existing file loaded successfully.”	Pass
TC-02	Start with no JSON	“File not found. Creating a new one...”	Pass (logic in try/except)
TC-03	Add valid student marks 85	Success; grade B stored	Pass

TC-04	Marks = abc	"Invalid input! Please enter an integer."	Pass
TC-05	Marks = 150	"Invalid marks! Range should be (1-100)."	Pass
TC-06	View three sample rows	Aligned table, grades A/C/D	Pass
TC-07	Update Himanshu to 78	"Score updated successfully."; grade C	Pass
TC-08	Update unknown name	"Student not found."	Pass
TC-09	Export CSV	File created; header + rows	Pass
TC-10	Menu choice 9	"Invalid Input"; menu repeats	Pass
TC-11	Menu choice abc	"Please enter a valid number."	Pass
TC-12	Exit option 5	"---Thanks for using---"; process ends	Pass
TC-13	View then try another option	Session should continue	Fail — option 2 breaks the loop
TC-14	Name case mismatch on update	No match if spelling/case differs	Pass (case-sensitive by design)

TC-13 is an accepted defect relative to FR-02 (the menu should remain available). It is recorded honestly rather than hidden, and is listed under Future Enhancements.

12. Challenges Faced

11. File-not-found on first run: solved by catching `FileNotFoundError` and initialising `record = []` rather than crashing.
12. Users typing words where integers are required: solved with `try/except ValueError` around both the menu and the marks prompts.
13. Keeping grades consistent after an update: solved by always calling `calculate_grade` on the new marks instead of editing the letter by hand.
14. Readable tables in a console: solved by delegating layout to `pandas.DataFrame.to_string` instead of manual spacing.
15. Sharing data with spreadsheet users: solved with `DataFrame.to_csv`, accepting that CSV is a snapshot and JSON remains canonical.
16. Update lookup by name: simple to code, but brittle (case, spelling, duplicates). This remains a product limitation.
17. View exiting the program: the `break` after option 2 was convenient while debugging display, but it conflicts with a looping menu. It is documented rather than silently "fixed" in this report, because the report describes the submitted source.
18. Working directory vs file paths: relative names mean the program must be started from the project folder; otherwise a second `empty_record.json` appears elsewhere.

13. Conclusion

The Student Grade Management System meets the mini-project goal: a working, file-backed Python application that adds students, computes grades from a single rule table, lists records, updates marks, and exports CSV. Architecture is small by design — one class, one menu, two files — which keeps the design diagrams, test cases and source in obvious correspondence.

Execution results confirm the happy paths (add, view, update, export, exit) and the defensive paths (bad marks, bad menu input, unknown student). The main functional gap is that viewing records terminates the session; uniqueness of IDs and delete/search operations are also absent. Those gaps do not prevent classroom use of the current build, and they define a clear next increment.

14. Future Enhancements

- Change option 2 so that viewing records returns to the menu instead of breaking the loop.
- Look up students by Unique ID and reject duplicate IDs on insert.
- Add delete-record and search/filter (by grade band or name substring, case-insensitive).
- Move validation into the class and return error codes/exceptions instead of printing from domain methods.
- Replace the global list with a repository object to make the class testable without touching disk.
- Add pytest cases for `calculate_grade` boundary values and for JSON round-trips.
- Optional Tkinter or web (Flask/Streamlit) front end on the same JSON store.
- Simple login or at least a confirmation before overwriting CSV and JSON.
- Support decimal marks and configurable grade boundaries via a config file.
- Correct the menu prompt so it reads [1-5] to match the five implemented options.

15. References

19. Python Software Foundation. The Python Language Reference and Standard Library (json, exceptions). <https://docs.python.org/3/>
20. The pandas development team. pandas documentation — DataFrame, to_csv, to_string. <https://pandas.pydata.org/docs/>
21. ISO/IEC 25010:2011. Systems and software Quality Requirements and Evaluation (SQuaRE) — used as a vocabulary for non-functional requirements.
22. Sommerville, I. Software Engineering. Pearson — requirements, architecture and UML-style design views.
23. IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications — structure of functional vs non-functional statements.
24. Project source: main.py, requirements.txt, README.md, record.json, Record CSV.csv — Student Grade Management System (local project folder).

25. Related public repository referenced in the project README: https://github.com/nkk-854/Vityarthi_Py_Project

Appendix A — Grading Scale

Marks (inclusive)	Grade	Interpretation
90 – 100	A	Outstanding
80 – 89	B	Very good
60 – 79	C	Good / average-plus
40 – 59	D	Pass
Below 40	F	Fail

Appendix B — Project File Structure

Student Grade Mangement System/

- main.py — application source (entry point)
- requirements.txt — pandas>=2.0.0
- README.md — setup and usage notes
- record.json — persistent JSON store (generated / updated at runtime)
- Record CSV.csv — CSV export (generated by option 4)
- Project_Report/ — this report, screenshots and diagrams

End of report.